# Take-home assignment: Technical Product Manager — Claims + Ledger Module

## Context

We're building a **modern, cloud-native, multi-tenant dental PMS**. This assignment focuses on one of the most complex and high-impact areas: **insurance claims and the financial ledger**.

We're looking for strong product judgment, system thinking, and technical depth. There's no single "correct" solution—what matters is that your solution is **coherent, implementable, and defensible**, and that you can explain tradeoffs clearly.

---

# Problem to solve

Design v1 of a **Claims + Ledger module** for a dental PMS.

The module should support:

- Creating insurance claims from procedures/treatment data
- Submitting claims (direct payer or clearinghouse-style integration) *(consider multiple clearinghouse, patient self pay)*
- Tracking claim statuses and payer responses
- Receiving remittances (ERA/EOB) *(dropboxes)*
- Posting payments, adjustments, write-offs, denials, reversals
- Maintaining an accurate **patient and insurance ledger** with strong auditability *(patient, guarantor relationship and ledger)*
- Producing reliable balance views and reconciliation hooks

Assume:

- The PMS is **multi-tenant** (many practices; potentially multi-location orgs) *(consider multi-state too)*
- We operate in the US dental billing context
- High correctness + auditability is required
- Integrations may be unreliable (retries, duplicates, delays)

---

## What to submit

## 1) Product + workflow design

Describe the main user workflows and decisions:

- Who are the primary users? (front desk, biller, office manager, etc.)

Primary users and respons

| Primary user | responsibility |
|---|---|
| Front desk | Scheduling, checkin, eligibility, collecting copays, verifying demographics, initiating claims<br>In smaller practices: does the responsibility of Internal RCM team as well |
| Dental assistant | Document support, CDT code capture |
| Dental Hygeniists | Document preventive procedures, radiographs etc |
| Dentist | Procedure confirmation |
| Practice Owners | AR, payer performance oversight |
| Internal RCM team (DSOs/Large org) | High volume claim submission, denial management, AR follow-up, posting.  Overall RCM tasks |

Probable non-obvious users

- Referring dentists
- Patients
- Labs
- Integrated systems - EHRs etc
- Legal and compliance
- Regulators
- Payment plan providers
- Outsource billing vendor (small to medium practices)
- Collection agency (may not have access to the application)
- Auditors

- Core workflows for v1 (minimum):
  - Create claim from procedures
  - Submit claim + track status
  - Receive ERA/EOB and post (auto + manual)
  - Handle denials/partials/reversals
  - Update patient responsibility and balances
- Include:
  - Scope for v1 vs later

- Success metrics
- Key edge cases you intentionally handle in v1
- Key edge cases you intentionally defer (and why)

| Roles/System | Tasks | | | | | |
|---|---|---|---|---|---|---|
| Patient | Schedules appt | After eligibility and provider network verification, cost estimates and consent vs selfpay | Patient receives statement and pays or follows up | Paid in full by due date | | |
| Frontdesk | Confirms appointment details and captures demographics | Check patient's eligibility and benefits | Payor credentialling | | | |
| RCM Team/Front desk | Reviews denial/underpayment and determines next action like appeal/resubmit or bill patient | Payor credentialling | | | | |
| Payor system Eligibility verification | Checks eligibility and benefits | | | | | |
| Discovery | When eligibility verification fails with the available data | | | | | |
| Payor system Provider network | Checks if the provider is in network | | | | | |
| Clinic | Perform procedures | | | | | |
| coding | Assign procedure code and diagnosis code | | | | | |
| Billing system | Generate billing claims<br>Review for completeness<br>Billing corrects data if necessary | Submit billing to clearing house | Generate patient statement | Resubmit claims | Billing posts patient payment, write-off and closes account | Preauthorization when needed |
| Clearing house | Submit claims to primary, secondary and tertiary payers | Routes the claims status from payor to Billing System | | | | |
| Payor | Adjudicates payers | When claim not approved, sends denial/partial payment and EOB | When approved external payer sends ERA/EOB and payment | | | |
| Collections | Sends reminders and follows up | Arranges payment plan | Sets up payment plan and collects installment | Escalates delinquency and notifies practices | | |
| Practice | Approves write-offs or refers to | | | | | |

| | | | |
|---|---|---|---|

V1:

1. Patient scheduling
2. Patient registration - include patient demographic, insurance card (if insurance card is uploaded reduce manual effort)
3. Eligibility and discovery
4. Provider network
5. Procedure code
6. Diagnosis code (primary diagnosis code)
7. Fee Schedule
8. Payor management
9. Place of service
10. Selfpay patient consent
11. Billing generates claims and checks for correctness
12. Duplicate validation at claim level and line level
13. Billing submits the claims to primary, secondary and tertiary
14. COB
15. collections

## 2) System design (services, boundaries, and data)

Propose an implementable architecture:

Where the “source of truth” lives for:

- claim status
- posted financial facts
- patient balances shown in the UI

**Service boundaries (e.g., Claims, Ledger, Payments/Posting, Integrations, Audit)**

- Insurance eligibility service:
  - Responsibilities
    - Coverage verification - Real-time eligibility checks, Batch eligibility, Payor responses

                - Patient-Plan relationship and coverage status including COB
                - Benefit interpretation
                - Eligibility history and audit trails - should maintain the last verification timestamp, retain the raw payor response, and who initiated the check
                - Integration with payers, clearinghouse, and PMS
                - Note: The vision is not just a 270/271 wrapper but a benefit intelligence engine to power treatment planning, cost estimates
    - Discovery service (This can be moved to V2)
        - Responsibilities
            - Eligibility routing to the discovery vendor
            - Normalization of data
            - Downstream Integration
            - Confidence scoring
    - Payor service
        - Responsibilities
            - Directory and metadata management
            - Payor contracting
            - Payor-specific rules
            - Clearing house routing rules
            - Payor rules engine
    - Documents and imaging service
        - Responsibilities:
            - Document capture and storage
    - Preauthorization service
        - Responsibilities
            - Preauthorization request service
            - Preauthorization response service
    - Billing Service
        - Responsibilities
            - Fee schedule determination
            - Provider and network validation
            - Place of service
            - Claim creation
            - Claim submission
            - Claims response

This will be the source of truth for all claims-related transactions.  This service must maintain the status history, timestamps, and related data.

    - Payment and posting service
        - Posting rule
    - Ledger service
        - This will be the source of truth for posted financial service.
        - Payments, adjustments, write-offs, denial, reversal, refunds all become ledger entries.  These Entries are never overwritten

        - Maintain a materlized view for patient balances in the UI. for example Patient balance, claim balnce, account balance etc. These entries can be updated
    - Integration service
    - Audit and events service

How data flows between them (sync calls vs events)

Sync calls:

- Validate demographic/insurnace
- Payor Preauthorization requirements
- Save procedure
- Validate CDT/Provider/POS
- Parse ERA
- Build claims
- Submit claim to Clearinghouse
- Creatae ledger endtires
- View updated balance
- Update Patient/Insurance
- Complete procedure

Event calls

- Emit Elibiligity verivcation
- Emit procedure completed
- Emit charge created
- Emit paymentposted
- Emit ClaimsSubmitted
- Emit ClaimStatusUpdated
- Emit ledger entry created

**Multi-tenancy (required):**

- Explain your tenancy model and how you prevent cross-tenant access
- Mention how you'd handle org → multiple practices (if relevant)

Two preferred tenancy options are

1. Multi-tenant shared applications with isolated databases
    a. Pros: Strong isolation, no cross-tenant leakage risk,
    b. Cons: Maintenance overhead
2. Hybrid Tenancy - Shared scheduling, charting etc, RCM isolated, and separate warehouse

a. Pros: scale RCM independently. PSM maintainability is easier
b. Cons: Complex architecture

Of these two I prefer hybrid tenancy for v1 with an approach to get to Multi-tenant in V2 or later.

To prevent cross-tenant access, recommendation will be:

- ➔ Each tenant have their own databases and no tables will be shared
- ➔ API routing is tenant scoped
- ➔ Connection pool will be tenant scoped
- ➔ Auth token should include tenant id, permission, role and location id (if needed). And backend must not accept tenant id that are not from token
- ➔ Domain layer authorization policies
- ➔ Event bus isolation
- ➔ Separate Redis databases or queues must be tenant scoped
- ➔ Analytics would have a centralized warehouse

An org can have any number of practices. The practices share the same tenant-scope RCM DB with PracticeID Boundaries

- ➔ Inside Org's DB, we enforce:
  - ◆ OrgID
  - ◆ PracticeID
  - ◆ LocationID (where there are multiple location)
- ➔ In this scenario
  - ◆ Users like RCM team for a large DSO, may be able to get an overall view of all RCM related operations
  - ◆ And practice level accesses like front desk would be able to get access only to the patient's information relavant to that location
- ➔ Creating org level and practic level accesses

**Ledger (required):**

- Choose a ledger approach and justify it:
  - double-entry vs single-entry
  - immutable ledger entries vs mutable transaction rows
  - how you represent reversals/voids/corrections without destroying audit history
- Explain how balances are derived (computed vs stored vs hybrid)
  - Double entry, immutable and hybrid will be my preferred approach
  - For correctness (eg, ledge entries are source of truth) and accessing balances (eg performance for quick access on the UI) hybrid model is better approach for ledger

    - Since single entry model is an audit red flag especially in an RCM system where adjustment can sometimes happen even after years from date of service and for legal reasons.

## 3) Data model (entities + key fields)

List your core entities and key fields, and how they relate. Include at least:

- Claim, ClaimLine
- Procedure / ServiceLine (or whatever you assume generates claims)
- Submission (attempts), ClearinghouseTransaction (if applicable)
- Remittance (ERA/EOB), RemittanceLine
- Payment, Adjustment, WriteOff, Reversal/Void
- LedgerEntry (or JournalEntry/Line if double-entry)
- PatientAccount (or AccountBalance projection)
- AuditLog / ChangeHistory

Include:

- Example relationships
- Key indexes / uniqueness constraints (especially for idempotency + concurrency)

## 4) APIs / contracts

Provide your proposed API surface (REST or RPC), including:

- Create claim (from procedures)
- Submit claim
- Update claim status (from integration callback / polling)
- Import remittance (ERA/EOB) (file upload and/or integration callback)
- Post remittance (auto post + manual corrections)
- Manual ledger entry / adjustment
- Fetch patient balance + ledger view

For each:

- Example request/response
- Idempotency strategy
- Top error cases and how you represent them

**Create claim (from procedures)**

**Request should include minimum the following:**

{

```
"transaction_set": "837",
"transaction_type": "dental",
"submitter": {
  "name": "ABC Dental Clinic",
  "identifier": "1234567890",
  "contact": {
    "name": "Jane Smith",
    "phone": "555-123-4567",
    "email": "billing@abcdental.com"
  }
},
"receiver": {
  "name": "XYZ Insurance",
  "identifier": "9876543210"
},
"subscriber": {
  "member_id": "S123456789",
  "first_name": "John",
  "last_name": "Doe",
  "dob": "1980-05-15",
  "gender": "M",
  "address": {
    "line1": "123 Main St",
    "city": "Seattle",
    "state": "WA",
    "zip": "98101"
```

```
  }
},
"patient": {
  "relationship_to_subscriber": "self",
  "first_name": "John",
  "last_name": "Doe",
  "dob": "1980-05-15",
  "gender": "M"
},
"billing_provider": {
  "npi": "1122334455",
  "name": "Dr. Emily Carter, DDS",
  "address": {
    "line1": "456 Dental Ave",
    "city": "Seattle",
    "state": "WA",
    "zip": "98102"
  },
  "tax_id": "12-3456789"
},
"claim": {
  "claim_number": "CLM-10001",
  "place_of_service": "11",
  "total_charge_amount": 250.00,
  "service_lines": [
    {
```

```
      "line_number": 1,
      "procedure_code": "D1110",
      "description": "Prophylaxis - Adult",
      "tooth_number": null,
      "date_of_service": "2026-05-15",
      "charge_amount": 100.00,
      "units": 1
    },
    {
      "line_number": 2,
      "procedure_code": "D0120",
      "description": "Periodic Oral Evaluation",
      "tooth_number": null,
      "date_of_service": "2026-05-15",
      "charge_amount": 150.00,
      "units": 1
    }
  ]
 }
}
```


## 5) Reliability, idempotency, and reconciliation

This section is critical. Cover:

How you handle duplicate inputs:

- **Same ERA imported twice**
    - If two or more 8335s have the same claim number, date, and payment amount, then that charge may be a duplicate
        - In this case, skip ingestion and log as a duplicate
    - Check each CLP
- **Duplicate clearinghouse callbacks**
    - This can happen due to duplicate submission, system errors or incorrect routing
        - Easiest approach is to establish a process in claims response service to enable unique claim update sequence number by checking claim#, NPI, date of service, claims lines
        - Automated claims scrubber to detect and dedupe before claims response service
        - Setting up status
        - Overall hash-based deduping, event-level idempotency for callbacks and idemptency keys for all submission and status operation.
- Concurrency:
    - Two users are posting at the same time
    - partial failure while posting a multi-line remittance
        - When 2 users attempts to post at the same time, the first transaction to commi\t increments the version.and the 2nd detects the mismatch. The design here could be that every acting include an idempotency key like: ERA_id+Claim_ID+Line_number
        - The design should also prevent ledger from being updated twice
        - For partial failure, when posting a multi-line remittance - for example if 3 out of 5 lines were updated and 2 did not update correctly, the system should be allowed to committed linked as successful. However, the lines that were not updated must be flagged. There should be systemmatic approach for reprocessing the remittance and applying the flagged lines using the idempotency keys. This guarantees no duplicate and avoids rolling back the entire remittance. This is a better approach especially when there are claims with large number of lines.
- Reconciliation:
    - how you detect “drift” between stored balances and ledger truth
    - what monitoring/alerts you’d put in place
    - basic strategy for repair (backfill/recompute/replay)
        - Ledger is the source of truth
        - There should be a periodic job that recomputes balances from the ledger and compares with stored aggregates. Any intolerance must be logged as drift (preferabbly with ID, delta and last change timestamp)
        - For monitoring there should be drift metrics, error budgets, event logs and audit trails

        - For repair strategy, recompute is done when the drift is detected and recompute id done directly from immutable ledger.
        - If the drift is from partial failure similar to the example above, replay the event using the idempotency keys.  This regenerates the correct entries
        - Backfill is done due to incorrect data or a logic bug.  Here, compensating entries are made to the ledger, preserving the audit trail.

## 6) Rollout plan

Describe how you'd ship this safely:

A RCM launch's rollout should be a small batch/progressive exposure, not a big bang cutover. The system must be controlled so that functions can be released gradually

- Feature flags strategy
    - The proposed approach of feature flags and claim release strategy
        - The following services/subsystems should be flaggable
            - Eligibility and benefit verification
            - CDT coding
            - Fee Sched application
            - Claim creation
            - Claim submission
            - ERA
            - Statement
            - AR
            - Denial and appeals workflows
            - Payment posting
            - Duplicate validation
            - Coordination of benefit
        - These flags should be per org,  per clinic, and per location.  At least per org for V1
    - Pilot Strategy:
        - Pilot strategy would include identifying the possibility of submitting low volume at launch.  The strategy could be any or a combination of these
            - Identify a payor or a few payors with very low volume, good adjudication process, and/or predictable work flow
            - Review the pilot results and success rate over a period of time
            - If the throughput and success rate meets threshold, release smaller payors in batches for a period of time
            - Gradually, increase to national payors like Delta Dental, Cigna, etc
            - If BCBSA payor has location-specific payor rules, begin releasing them in the next cycle
            - My recommendation is to release the government claims, like Medicare, Medicaid, and Veterans Affairs, last. The claims have

larger timely filing guidelines. And these payors have stricter guidelines on erroneous claims.

- **Data migration approach (if migrating from another PMS vs greenfield)**
  - **Migration from another PMS:**
    - There is a common assumption that claims systems are standardized across the industry and that data migration is a standard set of rules. Unfortunately, that isn't true. Every clinic, EHR system, and payor system stores and structures data differently, and each organization often has its own unique operational and reporting needs.
    - **Key edge case included in V1:** In some scenarios, it is necessary to retain historically closed claims. This is critical for duplicate-claim validation, frequency limits, active legal or audit inquiries, and claims associated with accidents, particularly those involving minors.
    - **Key edge cases you intentionally defer:** Even if there is no legal or compliance requirement to do so, it is still recommended to obtain a full export of the data warehouse. A full migration isn't necessary, but having the datadump provides valuable historical reference. Since the clinic can still meet its legal and audit obligations with the prior billing vendor for 6-7 years after the migration.
    - The most effective approach is to surface these gaps early in the product roadmap. Identifying data-model differences, workflow variations, and integration constraints up front to plan the right migration or export strategy.
    - Note: Migration has historically been the critical path, most complex, and high-risk component to RCM change.
    - This approach requires resource bandwidth
  - **Greenfield:**
    - This approach significantly reduces the implementation complexity by avoiding early heavy_lift migration
    - Streamlines onboarding for new clinics
    - Good for clinics that prefer to complete migration only after the RCM engine is stable and the core workflow is validated in production
    - Staff may end up working in 2 systems until the old AR runs off.

- **Validation plan in production (shadow posting, sampling, parallel runs)**
  - ***Challenges of validating in production or sandbox:***
    - Most clearinghouses and payors discourage sending non-production test claims into their production environments.
    - In many cases, a clearinghouse may offer a sandbox connected to the billing system, but it is not for payor systems

    - ***Shadow posting:***
        - Generate claims, but do not submit them
        - Payments should be calculated and not posted to the ledger
        - This validation of coding, pricing, and adjudication checks the logic and result, without impacting real revenue
    - ***Sampling:***
        - Establish the following sampling guidelines
            - 5-10 claims per payor group
            - High-risk claims
            - Coordination of benefits scenarios
            - Claims requiring attachments or supplementation documentation
    - ***Parallel runs:***
        - When possible, submit claims through both systems in parallel
        - Validate each step of the process by comparing results between the legacy and new system

- **What you would instrument from day 1**
    - Eligibility success rate
    - Claims creation success rate
    - Manual intervention before submission
        - # of manual edits per claims
        - %age of claim requiring manul edits
    - Clearinghouse rejection
    - FPAR by payer
    - Approval Cycle time
        - % of payments posted without manual intervention
    - Workflow timer

## 7) Assumptions + what you did not do

Include:

- Assumptions you made (3–10)
    - Member eligibility and discovery is retrieved on the day of the appointment and earlier before
    - Practices will house revenue cycle team.  PbN provides only SaaS solution
    - Payment portal is external to PbN, may have to be integrated with PbN for ledger
    - Uncouple claims from ledger module.  There may be customers who may want only claims billing
    - RCM team of large DSOs can process multi location
    - First pass claims acceptance must be around 80%
    - For dental claim medicare/medicaid has a longer timely filing guideline
- What you intentionally did not do (and why)

    - For tenancy models, the preferred approach is Multi-Tenant Shared Applicaiton, Isolated databases due to HIPAA compliance but if considering to scale RCM independently this may not be a preferred approach for V1, but definitely in long term this can be in a Vx.
    - NPI integration with the national registry
- If you used AI, tell us what parts you used it for (optional but encouraged)

---

# Evaluation criteria

We will evaluate:

- Correctness and real-world billing awareness
- Technical depth and architecture tradeoffs
- Strong multi-tenant thinking
- Financial rigor (auditability, reversals, reconciliation)
- Clear communication: can engineering actually build this?
- Good product judgment on v1 scope